Session Brief: Setting Up Your Project on Claude Code

30754 Artificial Intelligence in Strategic Management · Nov 4, 2026 · @Abhinav Pandey

Today's timeline

Clock	Block	Post in your group channel when it ends
0:00–0:40	Part A: Brainstorm	Five ideas, a line each; the one you chose; its five-sentence theory
0:40–1:00	Part B: Read two CLAUDE.md files	Three things the sources agree on; two you will copy; one you will leave out, and why
1:00–1:30	Part C: Build the skeleton	The repo link; your tree as text; screenshots of your page and of `/health` in the browser
1:30–1:50	Part D: Write your CLAUDE.md	Your CLAUDE.md in full; the lines Claude Code's critique made you change
1:50–2:00	Part E: Check	The checklist, ticked, with one line on anything that is not

What to carry from the reading

Four ideas from the article you just read decide what you do in the next two hours. Before Part A, each of you writes three sentences, in your own words, on the first three; two minutes, no AI.

1. **Context is a budget, not a bucket.** The more tokens in the window, the worse the model recalls any one of them. The job is the smallest set of high-signal tokens that gets the behaviour you want.
2. **The system prompt sits at the "right altitude".** Not a brittle list of if-then rules, not a vague "be helpful". Specific enough to steer, loose enough that the model can use judgment. Your CLAUDE.md is a system prompt, so this is the standard it is held to.
3. **Good agents fetch just in time.** They keep pointers (file paths, names, links) and load content only when a step needs it. Claude Code drops CLAUDE.md into context up front, then uses search to find everything else. Folder and file names are signals it reads, which is why an empty but well-named skeleton is worth building today.

4. **Long tasks need a harness.** Compaction (summarise and restart), structured notes (a NOTES.md the agent writes to itself), and sub-agents that work in a clean context and return a short summary. Claude Code does all three; the file you write today is the part of that harness you control.

Part A: Brainstorm (40 min)

The brief: build a working tool that puts an LLM where it is strong and designs around where it is weak.

The grid. Every idea has to name one cell from each column, and the third column is where the project is won or lost.

LLMs are strong at	LLMs are weak at	How you mitigate it
Reading and summarising a lot of messy text, fast	Making things up with full confidence	Retrieval over your own material, with citations the user can click
Drafting in any style or format	Arithmetic, counting, dates	Have the model write code that computes; never let it do the sums
Extracting structure from mess (tables from emails, criteria from a job ad)	Being consistent from one run to the next	Structured output plus a validator; run it n times and compare
Applying a rubric to many items the same way	Knowing what it does not know	Force "I am not sure" as an allowed answer; route those to a human
Generating many alternatives cheaply	Checking its own work	A second model, or a test, that judges the first one's output
Writing and running code	Knowing anything after its training cutoff	Give it tools: search, a file, an API

Where to aim. A pipeline calls the model once and shows the answer. We would rather see one of these, or something of the same weight:

- **Synthetic investors**: a panel of model investors, each with a different thesis (growth, value, impact), that interrogates your pitch and tells you which assumption would make it walk away.

- **Synthetic judges**: a jury that scores submissions (resumes, pitches, essays, policy proposals) against a rubric, cites the exact passage behind every score, and is checked against human raters on a sample.
- **Synthetic customers**: twenty personas built from real reviews or interviews react to a price change or a new feature; you compare their answers to a real survey.
- **Socratic chatbots**: a tutor or coach in the spirit of Plato that never gives the answer; it asks what you are assuming and what evidence would prove you wrong.
- **Multi-agent orchestrators**: a manager agent that splits a question across specialist agents (market, finance, legal, tech), reconciles their answers and shows you where they disagree.
- **Critic pairs**: one model generates, a second attacks; the user sees the argument, not just the output.
- **Agents with tools**: the model decides what to look up or run next, acts, and verifies the result before it reports back.
- **Pre-mortem machine**: give it a strategy; agents write the ways it has failed twelve months from now, rank them, and you go test the top one.
- **Negotiation sparring partner**: plays the supplier, landlord or employer with a hidden reservation price, then scores the deal you reached and shows what you left on the table.
- **Competitor war room**: one agent per competitor reads their filings, job ads and news; a chair synthesises likely moves, each with its evidence.
- **Due diligence with a bear case**: agents gather public facts on a startup, every claim cited; a sceptic agent writes the case against investing.
- **Pitch-deck stress test**: reads a deck, lists every number in it, asks where each one comes from, and flags the ones nobody can source.
- **Research agent with receipts**: turns a question into searches and a report where every sentence links to a source; a checker agent flags the unsupported ones.
- **Contract red-liner**: flags risky clauses against your template, quotes the clause, and says which ones it is not sure about.
- **Data analyst in a sandbox**: plain-English question to code to result to chart; the model writes the code, the code does the arithmetic, and a check runs before you see the number.
- **Hiring screen with a fairness audit**: scores applications against a rubric, then a second agent swaps names and genders and reports whether the scores move.
- **Decision journal**: logs your beliefs with probabilities, asks for the evidence, nags you to update, and scores your calibration over the term.
- **Stakeholder council**: regulator, incumbent, startup and consumer agents debate a proposed rule; the output is where they disagree and why.

- **Story engine with a continuity agent**: a manga or game master that keeps characters, facts and state consistent across episodes because a second agent checks every new one against the record.
- **Adaptive tutor**: remembers what you got wrong in a memory file, generates practice on exactly that, and shows your progress.

The list shows the range; it is not a menu. Start from a decision one of you actually faces. More to mine: 20 hackathon ideas built on agents and MCP and Masai School's beginner-to-advanced list.

A good idea also leaves room for what is coming: retrieval with citations (tomorrow), an agent with tools (11–12 Nov), and tests of the agent's work (18 Nov). If your idea has no place for any of those, it is too small.

Post after Part A. Five ideas, a line each; the one you chose; and its theory in five sentences: *We believe [users] will [get value] because [strength]. This breaks if [weakness] shows up, so we [mitigation]. We would know it works if [measurable test].* Run the paragraph past Plato before you post it; it will ask what has to be true for it to hold.

Part B: Read two CLAUDE.md files (20 min)

First, what you are reading. Claude, the model, has no tools, no memory and no loop. Claude Code is a *harness*: the software around the model that calls it, runs the tool it asked for (read a file, run a command, search), feeds the result back, and repeats until the task is done. The harness also compacts the conversation when it gets long, enforces permissions, and can spin up *sub-agents*, each with its own clean context, and collect their summaries. Managing several agents at once is what a harness is for.

On every turn the harness assembles the system prompt the model sees. **CLAUDE.md is the part of that system prompt you write.** It is loaded at the start of every session, so it is the standing orders for every agent that works in your repo. That is why it gets the "right altitude" test from the reading, and why Boris Cherny keeps his to about 2,500 tokens.

The two authors, and a warning. Search "Boris Cherny CLAUDE.md" or "Karpathy CLAUDE.md" and you will find viral gists with those names on them. Neither man wrote those files. This is the course in one lesson: a fluent artefact is not a verified one. Read the real sources below.

Who	What to read	Why it matters
Andrej Karpathy, his own file	karpathy/llm-council/CLAUDE.md, 166 lines	The project memory of a real, 99% vibe-coded app: what each file does, which port, which gotchas, which design decisions and why, ideas for later. Written for a reader with no memory of last session.
Andrej Karpathy, distilled by Jiayuan Chang	andrej-karpathy-skills/CLAUDE.md, about 65 lines, 228k stars	Four behaviour rules (think before coding, simplicity first, surgical changes, goal-driven execution), each tied to a failure Karpathy described on X. No project facts at all.
Boris Cherny, creator of Claude Code	His thread on how he uses Claude Code (Jan 2026); InfoQ's summary if X is blocked	His team's CLAUDE.md is in git and about 2,500 tokens. It holds style conventions, design guidelines and a PR template, and grows one rule at a time: whenever Claude gets something wrong, they add it. After every correction he ends with "update your CLAUDE.md so you don't make that mistake again."

How to read them. Six minutes each, then eight minutes to write your list.

1. Karpathy's own file: mark every line that only makes sense for *this* project (ports, file names, a bug he hit). That is project memory.
2. The distilled file: mark every line that would make sense in *any* project. That is behaviour. Notice there is nothing project-specific in it.
3. Boris's thread: find the loop, mistake → rule → file. Note what he keeps out of the file (he does not customise Claude Code much) and what he puts in slash commands instead.

Post after Part B. Three things all three sources agree on; two things you will copy into your own CLAUDE.md; one thing you will deliberately leave out, and why. A line each is enough.

Part C: Build the skeleton (30 min)

By 1:30 your group has a git repository with your own structure in it, almost every file empty, a Python backend that starts, and an HTML page that opens in a browser. No features yet.

Install and start (5 min, one laptop per group). Follow the setup page: on macOS run `curl -fsSL https://claude.ai/install.sh | bash`, on Windows PowerShell run `irm https://claude.ai/install.ps1 | iex`. Check with `claude --version`. Make the project folder, `cd` into it, run `git init`, then `claude`. The first launch opens a browser login; log in with your Claude account (Claude Code is not on the free plan).

The structure is tokens. Claude Code never sees your project; it sees text. Every folder name, file name and first-line comment is a token in its context, and the article you read says those names are the signals it uses to decide what to open and where to put things. A clear structure lowers the probability that the harness guesses wrong: puts the model call in the wrong file, invents a second entry point, edits a test instead of the code. A messy one raises it. So design the structure before the agent touches anything, and write it down. A suggested shape:

```
<project-name>/
├── CLAUDE.md        standing orders for the harness (Part D)
├── README.md        your theory, nothing else
├── .env             keys; never committed
├── .gitignore
├── .claude/         settings, agents, skills
├── backend/         Python: the API, and one file that talks to the model
├── frontend/        HTML, CSS, JS: the page the user sees
├── data/            your own material for retrieval
└── tests/           one test per feature, written first
```

That is a suggestion, not a spec. Split, rename or add folders to fit your idea: a multi-agent orchestrator may want `agents/` with one file per role, a synthetic judge may want `rubrics/`. The test is that a stranger, or a model, can read the tree and guess what each part does. Three things are fixed: `.env` and `.gitignore` exist from the first commit; one backend process serves both the page and the API on one port, so there is no cross-origin setup to fight; and there is a `/health` endpoint the tests can hit. "Empty" means each file exists, its first line says what it will do, and the backend starts.

Keys. `.env` holds one line, `OPENAI_API_KEY=` followed by the key AP gives you in class, plus `PORT=8000`. The backend reads it with `python-dotenv`. The key is shared by the whole class, so three rules, no exceptions: never paste it into Claude Code, any chat, or your Teams channel; never commit `.env`; and tell the harness it may not read the file. That last one is the `settings.json` content, from the official docs:

```
{
  "permissions": {
    "deny": ["Read(./.env)", "Read(./.env.*)"]
  }
}
```

A `.claudeignore` file does nothing; the deny rule is the mechanism. Deny rules win over everything, including bypass mode.

How to build it with Claude Code (20 min).

1. Press Shift+Tab until the status line says plan. In plan mode Claude reads and proposes; it edits nothing until you approve.
2. Paste your tree, say what each folder is for, and name the few files that must not be empty. Say explicitly: do not write CLAUDE.md, and do not add features we did not list. Read the plan. If it adds a database, a login page or a second framework, send it back.
3. Approve. Watch what it does. The driver reads every command before accepting it; a teammate checks the result against the tree you wrote.
4. Do not run `/init`. It would write a CLAUDE.md for you, and that is your job in Part D.

Check it (5 min). Run these yourselves, in a second terminal, not through the agent:

```
python -m venv .venv && source .venv/bin/activate      (Windows:
.venv\Scripts\activate)
pip install -r requirements.txt
uvicorn backend.main:app --reload
```

Open http://127.0.0.1:8000 and see your page; open http://127.0.0.1:8000/health and see `{"ok": true}`. Run `git status` and confirm `.env` is not listed. Then make your first commit.

Git, the short version. Your GitHub handout covers the rest; this is enough for today. If `git --version` fails, install Git (on Windows pick Git for Windows; Claude Code uses it too). Then, in the project folder:

```
git add .                                stage every file (.env stays out:
.gitignore)
git commit -m "Project skeleton"         save a snapshot with a message
git branch -M main                       name the branch main
```

To share it with your group: one person creates an empty repository on github.com (New repository, no README), then runs the two lines GitHub shows under "push an existing repository": `git remote add origin <url>` and `git push -u origin main`. Teammates run `git clone <url>` to get a copy. From then on, after every change that works: `git add .`, `git commit -m "what changed"`, `git push`. You may ask Claude Code to run these for you; read each command before you accept it.

Post after Part C. The repository link; your tree pasted as text; and two screenshots, your page open in the browser and `/health` returning `{"ok": true}`.

Part D: Write your CLAUDE.md (20 min)

Write it yourselves, in a text editor, with Claude Code closed. Aim for 40 to 80 lines; the official guidance is to stay under 200, because longer files reduce how closely the model follows them. Every line has to pass the test from the reading: specific enough to steer, loose enough to leave judgment to the model, and nothing the model could find by reading the code.

Six sections, in this order.

1. **What this is.** Three lines: the theory from your README, who the user is, what "done" looks like by 26 November.
2. **How to run it.** The exact commands from Part C: create the venv, install, start the server, run the tests. The agent will run these, so they must be right.
3. **Where things live.** One line per folder. `backend/llm.py` is the only file that calls the model. `frontend/` is plain HTML and JavaScript, no framework. `data/` is for retrieval material. `tests/` is where every feature starts.
4. **How to behave.** Five to eight rules, each tied to a mistake you expect. Pick from the distilled Karpathy file what applies: state assumptions before acting; the smallest change that works; touch only what the task needs; write the test first; when done, say how you verified it; record decisions in `NOTES.md`.
5. **What not to do.** No new frameworks, no database, no features nobody asked for, no editing a test to make it pass, never read or print `.env`.
6. **Mistakes we have seen.** Empty today. This is Boris Cherny's loop: whenever Claude gets something wrong, one line goes here, and every correction you give it ends with "update your CLAUDE.md so you don't make that mistake again."

What stays out. Anything the model can see by opening a file. Explanations of why you chose FastAPI. Anything that changes weekly, which belongs in `NOTES.md`. The key, obviously. Personal preferences ("talk to me in Italian") go in your own `~/.claude/CLAUDE.md`, which applies to every project on your laptop and is not shared with the group.

Test it (last 5 min). Save the file at the repo root, then start a fresh `claude` session so it loads.

1. Ask: *what does this project do, and how do I run it?* If the answer comes from your file without opening the code, the altitude is right.
2. Ask: *read CLAUDE.md; which lines would you ignore, which contradict each other, which are too vague to act on?* Fix the lines yourselves. Do not paste in its rewrite.
3. Ask it to print the contents of `.env`. The deny rule should stop it, and your file should already have told it not to try.
4. Commit.

Post after Part D. Your CLAUDE.md pasted in full, and the lines Claude Code's critique made you change. If it found nothing to change, say so, and say why you do not believe it.

Part E: Check and post (10 min)

Go through this list yourselves, in a terminal, not by asking the agent whether it is done.

- [ ] `uvicorn backend.main:app --reload` starts; http://127.0.0.1:8000 shows your page; `/health` returns `{"ok": true}`
- [ ] `pytest` runs and the one test passes
- [ ] `git status` is clean; `.env` is not in the repo; `.env.example` and `.gitignore` are
- [ ] A fresh `claude` session answers "how do I run this?" from CLAUDE.md, without opening the code
- [ ] Every member of the group has cloned the repo and can start the server on their own laptop
- [ ] Posted this checklist in your group's channel, ticked, with one line on anything that is not

What is due

Nothing from today is graded. Your five posts in your group's channel are what we read before tomorrow, and how we know which groups to sit with first.

Tomorrow you build retrieval over your own material and make it cite its sources. Before then, put the documents your tool will actually need (pitch decks, resumes, transcripts, reports, 5 to 20 files) in `data/`. If any of them are private, add `data/` to `.gitignore` instead of committing them.